INTERVIEW STUDY GUIDE · VOLUME 3

Development History, Validation, and Limitations

What changed from the original prototype to the current repository, which failures taught the important lessons, and what the retained evidence actually proves.

AM13E230 TINIElite Edge AI · Jainil Bavishi · 23 October 2026

Contents

1. Project evolution
2. How to interpret the original handoff
3. Major bugs, experiments, and lessons
4. Windows-to-WSL and repository migration
5. Verification strategy
6. Hardware results and performance
7. What is proven and how
8. Open limitations and future work
9. Interview-safe claims

1. Project evolution

> **HOW TO READ THE HISTORY TABLES**
> Each table contrasts an earlier state with a later, verified state. It does not mean every early experiment was wrong; early work was often necessary discovery. It means that claims must follow the final implementation and retained evidence, not an older prototype. For basic terms such as object code, Kconfig, and out-of-tree development, read Volume 0 first.

Phase A — Initial board and UART bring-up (July 2026)

- Created an AM13E23019 SoC description and LP-AM13E230 board definition inside an experimental Zephyr checkout.
- Reverse-engineered the dual-base UNICOMM layout from TI material.
- Implemented basic polling TX/RX and reached the first UART console output.
- Diagnosed missing pin mux and incorrect RX FIFO status logic.

Phase B — Generated model and NPU integration

- Inspected `mod.a` with Arm binutils to discover exported and unresolved symbols.
- Implemented NPU power/reset/privilege and the memory callbacks expected by the archive.
- Added NPU devicetree information and the initial execution loop.
- Learned that the generated runtime advances through hardware stages using a continuation function rather than one simple blocking accelerator call.

Phase C — First host protocol and automation

- Created a Python script to parse generated test vectors and quantize inputs.
- Built an early magic-header/fixed-payload/additive-checksum protocol.
- Experimented with a UART thread and message queue to separate reception from inference.
- Automated Windows build/flash/test through PowerShell and DSLite.

Phase D — Debugging and early profiling

- Fixed NPU access faults, NPU power sequencing, stack overflow, and UART data loss.
- Replaced millisecond timing with the hardware cycle counter.
- Attempted UART interrupts and persistent NPU SRAM caching; both were abandoned after hardware faults.
- Reported early latency values before the clock tree was independently read back.

Phase E — WSL-native reconstruction and public repository (October 2026)

- Reconstructed Python/west/toolchain dependencies inside Ubuntu 22.04 under WSL.

- Attached the XDS110 with `usbipd-win` and restricted udev permissions.
- Installed Linux TI UniFlash and proved native WSL erase/program/verify, replacing the earlier assumption that flashing must remain on Windows.
- Extracted project-owned changes into a clean out-of-tree repository with a pinned Zephyr manifest.
- Replaced the prototype transport with protocol v2 and retained JSON evidence.

Phase F — Reliability and evidence hardening

- Read back the clock tree, discovered the 32 MHz/200 MHz mismatch, implemented the real 200 MHz startup, and invalidated misleading early timings.
- Replaced NPU completion polling with verified IRQ 41 + semaphore completion.
- Added NPU-stage timeout, cold-reset recovery, and a separately clocked WWDT0 watchdog.
- Moved pin routing into Zephyr pinctrl and added UART receive error reporting.
- Added host unit tests, sanitized Zephyr-native ztests, static analysis, formatting, cross-build CI, symbol checks, and build artifacts.

Repository milestones

Commit	Purpose
`08104e2`	Initial hardware-validated out-of-tree demo.
`87d49bc`	Robust protocol and stress evidence.
`8493233`	Real 200 MHz clock initialization and validation.
`ee9293c`	Interrupt-driven NPU completion.
`644a4ae`	Bounded NPU timeout and reset recovery.
`e2eae52`	Independent hardware watchdog recovery.
`d35b301`	UART/pinctrl hardening and native tests.
`ab8927f`	Proper Zephyr library integration.
`a1f0873`	Python quality gates and robustness fixes.

2. How to interpret the original handoff

The original knowledge-transfer document accurately captures valuable discovery work, failed experiments, and the state of the early Windows prototype. It is not a description of the final repository. Use this correction table:

For example, the row saying “modifications live inside Zephyr” describes where experimental files once happened to be stored. The current result is an out-of-tree module: the same sort of board/driver support

lives in this project repository and is discovered by Zephyr through metadata. This makes a clean build reproducible and prevents hidden local Zephyr edits from becoming a dependency.

Earlier statement	Current reality
Multithreaded UART thread + `k_msgq`	The final server is intentionally single-request and synchronous. NPU completion still uses ISR/semaphore concurrency.
259-byte packet + additive checksum	Protocol v2 uses a variable-length header, sequence number, CRC-32, and structured responses. An inference request is 268 bytes.
NPU completion through CTL5 polling	Current completion uses physical DONE IRQ 41; polling is retained only as historical baseline evidence.
“282 µs pure NPU” / “340 µs hybrid”	Invalid as current latency because hardware ran at 32 MHz while software converted cycles as 200 MHz. Current recovery image: 554/129/701 µs.
“100% accuracy”	Incorrect scientific claim. Six fixtures matched exactly; no independent dataset exists.
DSLite is Windows-only	Linux UniFlash/DSLite 9.7 was installed and hardware flashing from WSL was verified.
Hardcoded COM9 and Windows paths	Current client discovers Linux serial devices or accepts `TINI_UART_PORT`; scripts use repository-relative paths and environment overrides.
Modifications live inside Zephyr	Current work is an out-of-tree module and manifest repository.
Several CPU/NPU model variants available	The maintainable public project contains only the active generated `mod.a`, header, and six fixtures. No equivalent CPU model is available.

3. Major bugs, experiments, and lessons

3.1 UART had no output

Cause: PA0/PA1 were not routed to the UART function. A peripheral can be correctly configured internally while remaining physically disconnected from pins.

Fix evolution: initial direct IOMUX writes proved the setting; the final implementation expresses pins in devicetree and applies them through Zephyr pinctrl.

Lesson: debug signal paths from peripheral register to package pin, not just driver code.

3.2 Receive always appeared empty

Cause: the code interpreted RX FIFO Full rather than RX FIFO Empty. The correct rule is: if RXFE is set, return “no character”; otherwise read RXDATA.

Lesson: status-bit names and polarity matter more than intuition.

3.3 UART interrupt attempt BusFaulted

Cause: interrupt register offsets were applied to the UNICOMM wrapper base rather than the UART base, producing an invalid MMIO access.

Decision: revert to the known polling API rather than pretend an unverified interrupt path worked. UART async support remains future work.

3.4 NPU access HardFault

Cause: the GSC peripheral protection configuration did not grant privileged Cortex-M33 access to the coprocessor memory window.

Fix: set the TINIElite privilege bit during driver initialization.

3.5 NPU never completed

Cause: power/reset control registers require unlock keys in their upper bits. Setting only the low enable bit did not perform the protected operation.

Fix: implement the TI-aligned reset/power sequence with keys.

3.6 Parameter SRAM caching experiment failed

Hypothesis: retain all model weights in NPU SRAM to avoid repeated loads.

Why it failed: parameter SRAM is 2 KiB and serves as stage-local scratchpad. The generated runtime swaps data according to its own schedule. Freezing prior content breaks later stages and can cause faults.

Lesson: optimize only after understanding hardware capacity and the generated runtime's ownership model.

3.7 Random inference HardFaults

Cause: the generated runtime needed more than the original small main stack for intermediate data.

Fix: configure a conservative 32 KiB main stack. This is safe for the known image but consumes a significant fraction of 96 KiB SRAM; stack watermarking is a future optimization.

3.8 UART bytes were lost or corrupted

Historical cause: an earlier console/interrupt arrangement competed for incoming bytes. The final driver exposes only the polling UART API, and the application owns the receive path.

Additional hardening: physical overrun/parity/framing/break/noise flags now become structured transport errors.

3.9 Additive checksum mismatch

Cause: signed versus unsigned byte arithmetic differed between Python and C.

Long-term fix: replace the entire prototype packet with explicit unsigned serialization and standard CRC-32.

3.10 Partial frame after serial reopen

Observed behavior: a missing final CRC byte was followed by the first magic byte of a later `READY`, so a naïve parser consumed part of the next frame.

Fix: the host parser now slides one byte after an invalid candidate instead of discarding the whole declared frame. A unit test recreates the exact boundary, and ten repeated reopen/fault/recovery sessions passed.

3.11 Timing looked excellent but was wrong

Cause: devicetree and Zephyr were told the CPU ran at 200 MHz, while clock read-back showed the MCU still at 32 MHz. Cycle conversion therefore divided by 200 MHz and understated elapsed time by roughly 6.25×.

Fix: implement the real clock sequence, add raw register telemetry, and mark all earlier timings superseded.

Lesson: metadata is not measurement. Verify clocks at hardware registers before making performance claims.

4. Windows-to-WSL and repository migration

4.1 USB forwarding

WSL 2 does not directly own Windows USB devices. `usbipd-win` binds the XDS110 on Windows and attaches it to the Ubuntu VM. The board appears as Linux USB devices and an ACM serial port. Binding persists, but attachment usually must be repeated after unplug/reboot.

```
# Elevated Windows PowerShell
& "C:\Program Files\usbipd-win\usbipd.exe" list
& "C:\Program Files\usbipd-win\usbipd.exe" bind --force --busid 4-3
& "C:\Program Files\usbipd-win\usbipd.exe" attach --wsl --busid 4-3
```

The actual BUSID must be copied from `list`; angle-bracket placeholders are documentation notation, not literal PowerShell syntax.

4.2 Linux permissions

A narrow udev rule grants the `dialout` group access to TI XDS110 runtime/firmware-update USB IDs rather than making every USB device globally writable.

4.3 Native flashing

TI UniFlash 9.7 for Linux supplies `dslite.sh`. `scripts/flash.sh` sets its library paths, uses the repository CCXML, and requests erase, program, verify, and unlock operations. Native WSL flashing was physically verified.

4.4 Clean workspace requirement

The old Zephyr checkout still contained legacy in-tree project edits, including another board with the same name. Building the out-of-tree module against that contaminated tree can create duplicate-board conflicts.

Reproducible builds use a fresh west workspace at the pinned revision. The legacy tree was preserved rather than destructively rewritten.

5. Verification strategy

No single test proves the complete system. The project uses layers of evidence:

Layer	Checks	Failure class caught
Artifact integrity	SHA-256 manifest for archive, header, vectors.	Accidental generated-artifact changes.
Python unit tests	16 tests for parsing, quantization, framing, CRC, recovery, result/status validation.	Host and protocol-reference regressions.
Native Zephyr tests	5 ztests with ASan and UBSan.	C CRC/validation bugs and memory/undefined behavior in portable protocol code.
Static quality	Ruff, formatting, Cppcheck, ShellCheck, syntax compilation.	Style, suspicious C patterns, script errors.
Cross-build	Clean pinned Zephyr + GNU Arm Embedded GCC 10.3.1.	Target compile/link/configuration problems.
ELF inspection	NPU kernels, continuation handler, wait API, watchdog, protocol, pinctrl symbols.	Silent omission of required integration from the linked image.
Flash verification	TI DSLite erase/program/verify.	Image not actually reaching target flash.
Hardware smoke	Six exact fixtures, IRQ counts, clock status.	Basic end-to-end board failures.
Hardware stress	600 inferences, JSON timing/results.	Repeatability and transport instability.
Fault injection	CRC damage, NPU timeout, watchdog deadlock, UART BREAK.	Error paths that happy-path tests cannot exercise.

5.1 Why sanitizers run on native_sim

ASan and UBSan are practical on a host process, where they can detect out-of-bounds memory access and undefined behavior with rich diagnostics. They do not simulate AM13E230 peripherals, so physical hardware tests remain necessary.

5.2 Why symbol checks matter

A successful link alone does not prove the intended accelerator path was retained. CI runs `arm-none-eabi-nm` and requires key model, interrupt, protocol, watchdog, and pinctrl symbols. Runtime IRQ evidence then complements this static proof.

6. Hardware results and performance

6.1 Current recovery-enabled image

Measurement	Result
Golden fixtures	6/6 exact output vectors.
Stress regression	600/600 exact results; zero retries.
NPU completions	5 per inference; 3,000 total.
Spurious NPU interrupts	0.
Accelerator wait	554 µs median.
Generated runtime/CPU portion	129 µs median.
Total inference	701 µs median.
Clock state	200 MHz MCLK; 100 MHz UART source; 3 flash wait states.
Firmware footprint	25,492 bytes flash; 43,616 bytes RAM.

6.2 Polling versus interrupt completion

Version	Wait / NPU	Runtime	Total	Main-thread interpretation
Clock-validated polling baseline	510 µs median	128 µs	668 µs	Main thread actively runnable for at least 76.3% of inference.
IRQ + semaphore	554 µs	117–129 µs	689–701 µs	Main thread blocked during NPU waits; ≤21% wall time remains for non-wait main-thread work.

Interrupt completion increased latency by about 33 µs (4.9%) in the direct comparison. That is acceptable because the goal was bounded, schedulable completion—not claiming interrupts are always faster. Exact whole-system CPU utilization and energy were not measured.

6.3 Fault-recovery evidence

- Deliberately corrupted CRC was rejected and the next request succeeded without reset.
- Deliberate NPU non-start returned `-ETIMEDOUT`, cold-reset, restored `READY`, then passed 600 inferences.
- Deliberate watchdog starvation caused reset cause `0x0e`, watchdog restarted, then passed 600 inferences.
- Physical UART BREAK produced structured transport error `8` / `-EIO`, followed by fresh readiness and successful inference.

7. What is proven and how

Claim	Evidence	Status
The firmware builds reproducibly.	Pinned manifest, clean local build, public GitHub Actions cross-build.	Proven
The generated NPU path is linked.	ELF symbols for TINIElite kernels and continuation handler.	Proven
Physical NPU completions drive progress.	IRQ 41 counters, host enforcement, 3,000 completions/zero spurious.	Proven
Six supplied cases reproduce golden outputs.	Exact three-byte comparisons on physical board.	Proven
The system runs at 200 MHz safely.	PLL/source/divisor/flash raw register read-back.	Proven
Transport recovers from corruption.	Deliberate CRC damage and UART BREAK tests.	Proven
Stalled accelerator and firmware recover.	NPU semaphore timeout/cold reset and WWDT0 reset-cause tests.	Proven
General waveform accuracy is high.	No independent labeled dataset.	Not established
NPU is faster/more efficient than CPU model.	No equivalent CPU artifact or power measurement.	Not established
Production readiness/certification.	No product safety/security/field qualification program.	Not claimed

8. Open limitations and future work

8.1 External inputs required

- **Independent accuracy study:** requires a representative labeled dataset and authoritative preprocessing definition.
- **Model regeneration:** requires source model, training pipeline, and compatible licensed compiler flow.
- **CPU comparison:** requires an equivalent CPU-target model artifact.
- **Power/energy:** requires current measurement equipment or board instrumentation.

8.2 Engineering improvements

- Implement and validate interrupt-driven or asynchronous UART with buffering.
- Measure stack high-water mark and reduce the conservative 32 KiB allocation.
- Add trace-based whole-system CPU utilization and interrupt-load measurement.
- Pin a redistributable container image digest for stronger toolchain reproducibility.

- Separate reusable AM13x support further and prepare upstream-quality Zephyr patches.
- Replace remaining raw clock/reset/power register logic with reviewed subsystem abstractions when platform support exists.
- Disable diagnostic fault-injection Kconfig options in a deployment configuration.

8.3 Known ABI limitation

The archive's memory callbacks return `void`. Bounds violations are prevented/logged locally, but a callback cannot return an error to the generated runtime. This is an interface limitation of the available binary artifact, not something that can be completely repaired without a compatible regenerated runtime or a separately checked fault channel.

9. Interview-safe claims

Safe and specific

- “I integrated a supplied TVM/TI model archive; I did not train it.”
- “I built out-of-tree Zephyr platform support and the surrounding inference system.”
- “Six supplied fixtures matched exactly across 600 repeated inferences.”
- “We verified five physical NPU completion interrupts per inference.”
- “The current recovery image measured 701 µs median on a read-back-validated 200 MHz clock.”
- “The design recovers from CRC damage, NPU timeout, UART errors, and complete main-loop deadlock.”

Avoid

- “I created or trained the neural network.”
- “The model has 100% accuracy.”
- “The whole graph runs only on the NPU.”
- “Interrupts made it faster.”
- “The firmware is production ready.”
- “Power consumption or NPU-vs-CPU speedup was measured.”
- Quoting superseded 282/340 µs values.

Continue with Volume 4 for ready-to-practice explanations, hard questions, STAR stories, and a live-demo checklist.